

OS Project Spring 2026

Guidelines

- **Group Size: 3 students (cross-section groups are also allowed).**
- You need to make a console-based application; if you include the graphics, it will be a bonus of two absolutes.
- There should be enough visualisations that demonstrate the project scope.
- **Plagiarism is strictly prohibited; the minimum punishment will be an F grade.**

Description

Simulate two neighbouring intersections (F10 and F11) with concurrent vehicles (threads), controller processes coordinating via pipes, priority handling for emergency vehicles, and an integrated Parking Lot System attached to the F10 intersection.

Each parking lot provides 10 parking spots and a bounded waiting queue, implemented using semaphores: one representing available parking spots and another representing available waiting queue slots. Vehicles that intend to park must interact with this subsystem safely, ensuring they do not block the intersection.

High-Level Scenario Two Intersections (F10 and F11)

F10 and F11 are close enough for vehicles to travel between the two sectors. Both intersections operate independently but coordinate when necessary (especially during emergency events).

Vehicles (15 default)

Each vehicle is represented as a thread and belongs to one of six categories:

- Ambulance
- Firetruck
- Bus
- Car
- Bike
- Tractor

Vehicles spawn at random intervals at various entry points and have a destination (straight, left, or right). Some vehicle types (Cars, Bikes, Tractors, Buses) may attempt to use the Parking Lot before crossing.

Traffic Controllers

Each intersection has its own controller:

- Implemented as separate processes for IPC demonstration

- Spawned via `fork()` from the parent simulation process
- Controllers exchange information via pipes

Priority Rules

- Ambulances and Firetrucks have the highest priority
- They can interrupt normal green/red cycles
- The system must clear the path in advance
- Buses may be given medium priority
- Cars, Bikes, and Tractors have normal/lower priority

Safe Crossing Constraint

Only non-conflicting movements may proceed concurrently.

Coordination Between Traffic Signals

If an ambulance is moving from F10 towards F11, and F11 to F10:

- F10 informs F11 via pipe or message
- F11 prepares by clearing its intersection
- This prevents blocking the ambulance's route

Parking Lot System

Each intersection (F10 and F11) includes an adjacent Parking Lot Module that simulates real-world parking behaviour under limited capacity and high traffic pressure.

Core Characteristics

1. Fixed Parking Capacity (10 spots)

Each lot contains 10 parking spaces, represented by a semaphore. No more vehicles can park than the available spots.

2. Parking Waiting Queue (Bounded via Semaphore)

A second semaphore represents the maximum number of vehicles allowed to wait for parking.

- Prevents unbounded waiting queues
- Ensures system stability under heavy load

3. Interaction with Intersections

Vehicles intending to park must:

- Reserve a parking wait-slot (or spot) before entering the intersection

- Never block the intersection while waiting to park
- Only request intersection access when they have confirmed parking availability

4. Priority Behaviour with Parking

- Emergency vehicles never interact with parking
- Parking logic must not interfere with emergency preemption
- Parking-bound vehicles may be temporarily delayed if needed to clear emergency paths

Required Features Vehicle Threads (Concurrency)

Each vehicle is a thread (pthread).

Vehicle metadata:

- id
- type (category)
- origin
- destination
- priority
- arrival_time

Graceful Shutdown & Cleanup

Simulation must stop cleanly after:

- A configurable number of vehicles
- Or on SIGINT (Ctrl+C)

Rubrics:

Category	Evaluation Criteria	Max Points	Student 1	Student 2	Student 3
System Architecture	Multi-Process Design: Correct use of fork() to create F10 and F11, controllers. Reliable bidirectional Pipe (IPC) communication for emergency coordination.	40			
Concurrency & Threads	Vehicle Simulation: Each of the 15 vehicles is a separate pthread. Proper storage of metadata (ID, Type, Origin, Destination, Priority).	30			
Parking Logic	Semaphore Management: Correct use of dual semaphores (10 spots + bounded queue). Verification that vehicles never block the intersection while waiting to park.	40			
Traffic Control	Priority & Safety: Effective implementation of priority rules (Emergency > Bus > Others). Non-conflicting movement logic (no collisions).	30			
Stability & Cleanup	Graceful Shutdown: System catches SIGINT (Ctrl+C), stops spawning, joins all threads, and deallocates all IPC/Semaphore resources cleanly.	20			
Viva (Project)	Implementation Defense: Individual ability to explain their specific code and concepts from project.	40			
Viva (Lab Concepts)	Theoretical Knowledge: This viva will be taken from all the concepts taught from all the labs we studied not necessary from project.	50			
TOTAL		250	/250	/250	/250